

次世代エンタープライズシステムにおける動的マルチエージェントアーキテクチャの研究報告書: IPA共通フレーム準拠と自己進化型AIの統合

1. エグゼクティブサマリー

本報告書は、現代のエンタープライズソフトウェア開発が直面する複雑性と技術的流動性に対応するための、包括的なアーキテクチャフレームワークを提案するものである。本提案の中核となるのは、アプリケーションアーキテクト (AA)、インフラストラクチャアーキテクト (IA)、および言語特化型のポリグロットコーダー (PC) という3つの役割を厳格に分離し、それらを統括する「メタアーキテクト」が状況に応じてエージェントを動的に生成・更新するという概念である。

このアーキテクチャは、日本のシステム開発標準である IPA 共通フレーム (SLCP) の第2階層 (システム開発工程) および第3階層 (アクティビティ) に準拠し、生成AIによる自律的な開発プロセスを、産業界で検証された厳格なプロセスモデルにマッピングすることを目的としている。

特筆すべきは、**Model Context Protocol (MCP)** を採用することで、エージェントと外部ツール (ファイルシステム、クラウドAPI、Gitリポジトリ等) との間の相互運用性を標準化し、特定のAIモデルベンダーへの依存を排除している点である。また、技術文書の更新を検知してエージェントの構成ファイル (AGENTS.md やシステムプロンプト) を自動的に書き換える「自己進化メカニズム」を導入することで、常に最新の技術スタック (例: Terraformの新バージョン、新しいJavaのLTSなど) に適応可能な「エージェントスワーム (群知能)」を実現する。

本報告書では、このアーキテクチャの理論的背景、IPAフレームワークへの詳細なマッピング、各エージェントの詳細定義、そして実装に必要なスキルマトリクスを15,000語にわたり詳述する。

2. 戦略的背景とアーキテクチャ哲学

2.1 現代ソフトウェアエンジニアリングの課題と動的アーキテクチャの必要性

従来のソフトウェア開発ライフサイクル (SDLC) は、静的な役割定義と固定されたツールチェーンに依存してきた。しかし、現代の技術環境は極めて流動的である。クラウドサービスのAPIは週単位で更新され、セキュリティ標準 (NIST、CISベンチマークなど) は頻繁に改定され、プログラミング言語のエコシステムは日々進化している。このような環境下では、特定のバージョンのJavaやTerraformにハードコードされた静的なエージェント定義は、瞬時に陳腐化する運命にある¹。

本提案が解決を目指すのは、この「固定化された知識」の問題である。解決策として自己進化型

エージェントシステム (**Self-Evolving Agent System**) を導入する。このパラダイムにおいて、エージェントは単なる作業者ではなく、それ自体が設定可能なソフトウェア実体として扱われる。上位の「メタエージェント」が技術環境を監視し、ワーカーエージェントをインスタンス化するための構成ファイル(システムプロンプト、ツール定義、制約条件)を動的に書き換えるのである³。これにより、システムは人間の手動介入なしに、「Rust 1.78 エキスパート」や「Terraform AWS Provider v5.0 アーキテクト」を即座に生成し、開発ラインに投入することが可能となる。

2.2 責任分掌の三原則: アプリケーション、インフラ、コードの分離

生成AIを用いた開発において最も懸念されるリスクの一つは、コンテキストの汚染による幻覚(ハルシネーション)の増幅である。複雑なビジネスロジックと、微細なインフラ設定を単一のエージェントに処理させると、推論能力が分散し、エラー率が指数関数的に上昇する。これを防ぐため、本アーキテクチャでは、高度に機能する人間によるエンジニアリングチームを模倣し、厳格な職務分掌(Separation of Concerns)を強制する⁵。

1. **アプリケーションアーキテクトエージェント (AAA):** システムの論理的ビューを担当する。データモデル、API契約 (OpenAPI/Swagger)、ビジネスロジックフロー、ドメイン境界 (Bounded Contexts) を定義する。このエージェントは、コードが「どこで」実行されるかには関知せず、コンポーネントが「どのように」相互作用するかに注力する⁵。
2. **インフラストラクチャアーキテクトエージェント (IAA):** システムの物理的・クラウドビューを担当する。ネットワークポロジ、セキュリティグループ、IAMロール、リソースプロビジョニングを Infrastructure as Code (IaC) を用いて定義する。AAAが定義した論理コンポーネント(例:「決済サービス」)を、具体的なクラウドリソース(例:「AWS Lambda関数とDynamoDBテーブル」)にマッピングし、セキュリティコンプライアンスを保証する⁷。
3. **ポリグロットコーダーエージェント (PCA):** 実装を担当する。これらはAAAによって定義された契約を満たし、IAAによって提供された環境内で動作するコードを記述するためにインスタンス化される、短命 (Ephemeral) なエージェントである。Python、Go、TypeScriptなど、タスクに最適な言語で実装を行う。

2.3 IPA 共通フレーム (SLCP) の役割と位置づけ

AIの活動を産業標準に接地させるため、本アーキテクチャは **IPA 共通フレーム 2013 (SLCP)** をオントロジーとして採用する。AIの振る舞いを、ブラックボックス的な「魔法」としてではなく、IPAの第2階層(システム開発タスク)および第3階層(アクティビティ)にマッピングされた具体的な「作業」として定義することで、監査可能性と予測可能性を担保する⁹。これにより、企業は既存の品質管理プロセスを大きく変更することなく、AIワーカーを組織に統合することが可能となる。

3. 動的エージェント生成のメカニズムと技術基盤

3.1 エージェントファクトリーパターン(エージェント・スミス・パターン)

本アーキテクチャの核心は、文献において「エージェント・スミス」パターン¹⁰と呼ばれる動的エージェントファクトリーにある。これは単一のエージェントではなく、親エージェントが特定のスコープと寿命

を持つ子エージェントを再帰的に生成するシステムである。

3.1.1 メタアーキテクト(頭脳)の役割

メタアーキテクトは、ファイルシステムおよび外部ドキュメントソースへのアクセス権を持つ上位のオーケストレーションエージェントである¹¹。その主な役割はコンテキストエンジニアリングとロールのインスタンス化である。

- 入力: ユーザーからの高レベルな要求(例:「決済サービスをGo言語とAWS Lambdaを用いてマイクロサービス化せよ」)。
- プロセス:
 1. 現在の技術スタックと照らし合わせて要求を分析する。
 2. 「Tech Radar(技術レーダー)」(承認された技術とバージョンのベクトルストア)を参照する。
 3. ワーカーエージェントのペルソナ、制約条件、使用ツールを定義する AGENTS.md または同等の構成ファイルを生成・更新する¹³。
- 出力: 以下の3つの特定のサブエージェントを生成(Spawn)する。
 - Agent_App_Pay_v2(決済ロジック特化)
 - Agent_Infra_Lambda_Go(AWSサーバーレス特化)
 - Agent_Coder_Go_1.22(Go言語シンタックス特化)

このプロセスにより、プロジェクトのニーズに合わせて最適なスキルセットを持つエージェントチームが即座に組成される。

3.2 自己進化のメカニズム: 反射的プロンプトエンジニアリング

システムは「反射的プロンプトエンジニアリング(Reflexive Prompt Engineering)」を通じて自己進化を達成する。新しいライブラリやセキュリティ標準がリリースされた際、メタアーキテクトはエージェント定義リポジトリに保存されているシステムプロンプト自体を更新する。

- 検知(**Detection**): 「テクノロジーウォッチャー」モジュール(mcp-jina-reader や Firecrawl などのMCPツールを使用)が、公式ドキュメントの更新(例: AWS CDK v2のリリースノート)をスキャンする¹⁵。
- 同化(**Assimilation**): メタアーキテクトはこの新しい知識を取り込み、インフラストラクチャエージェント用の instruction_set(指示書)を書き換える。
- 展開(**Deployment**): 次回、インフラストラクチャエージェントがインスタンス化される際、そのエージェントは「新しいCDKコンストラクトの知識を持った状態」で誕生する。これにより、エージェントフレームワーク自体のコードを変更することなく、労働力(Workforce)の知識アップデートが可能となる²。

3.3 Model Context Protocol (MCP) : エージェントの神経系

Model Context Protocol (MCP) は、エージェントとその環境を接続するユニバーサルバスとして機能する。これは、アプリケーションアーキテクト、インフラアーキテクト、およびコーダーの分離を実現する上で不可欠な技術基盤である¹¹。MCPは、エージェントが外部ツールと対話するための標準化

されたインターフェースを提供する。

- **Filesystem MCP Server:** エージェントがコードや設定ファイルを安全に読み書きすることを可能にする¹⁸。これにより、エージェントは自身の記憶領域としてファイルシステムを利用し、状態を永続化できる。
- **AWS/Terraform MCP Server:** インフラエージェントが、実際にクラウドAPIを叩いてリソースプロパティを検証することを可能にする。これにより、存在しないインスタンスタイプを指定するといった「幻覚」を防ぐ¹²。
- **Git/GitHub MCP Server:** エージェントがブランチの作成、プルリクエスト(PR)の提出、差分(diff)のレビューを行えるようにする²⁰。これにより、AIによる変更履歴の管理と人間によるレビュープロセスへの統合が可能となる。
- **Diagrams MCP Server:** アプリケーションアーキテクトが、システムの構造を可視化するための図(ダイアグラム)を生成することを可能にする²¹。これは人間による検証を支援するために極めて重要である。

4. IPA 共通フレームへの詳細マッピングとプロセス設計

本セクションでは、提案するAIアーキテクチャをIPA共通フレームの具体的なレイヤーにマッピングし、各フェーズにおけるエージェントの挙動を詳述する。これにより、AIのアクションが既存のシステム開発タスクとどのように整合するかを明らかにする。

4.1 IPA 第2階層: システム企画 (System Planning)

このフェーズでは、システム化の構想と計画策定が行われる。AIエージェントは、曖昧な要求を具体化し、実現可能な計画へと落とし込む役割を担う。

4.1.1 第3階層: 要件定義 (Requirements Definition)

- 従来のタスク: 機能要件および非機能要件の定義、業務フローの策定。
- AIエージェントワークフロー:
 - 要件分析エージェント (**Requirements Analysis Agent - RAA**): ユーザーとの対話を通じて曖昧さを排除する。このエージェントは、ルールベースおよび機械学習モデルを用いた「曖昧性検出 (Ambiguity Detection)」アルゴリズムを搭載しており、「高速な」、「安全な」といった主観的で曖昧な用語をフラグ付けし、具体的な数値目標 (SLA) や規格 (例: レスポンスタイム200ms以内、AES-256暗号化) への置換を促す²²。
 - 成果物: Markdown形式の要件定義書 (RDD)。ユースケース図、非機能要件リスト、用語集を含む。

4.1.2 第3階層: システムアーキテクチャ計画 (System Architecture Planning)

- 従来のタスク: ハードウェア・ソフトウェアの方針策定、システムライフサイクルの定義。
- AIエージェントワークフロー:
 - メタアーキテクト: プロジェクトに最適な「エージェントスタック」を決定する。例えば、要件定義書に「高並行処理」が含まれている場合、Erlang/ElixirやGo言語に特化したコーダー

エージェントを選択する。また、クラウドプロバイダー(AWS, Azure, GCP)の選定を行い、それに対応したインフラアーキテクトエージェントを待機させる。これは、プロジェクト専用の「開発チーム」を動的に編成するプロセスに相当する。

4.2 IPA 第2階層: システム設計 (System Design)

このフェーズは、アプリケーションアーキテクトとインフラアーキテクトの分離が最も重要となる局面である。論理設計と物理設計を明確に分けることで、技術的負債の発生を抑制する。

4.2.1 第3階層: アーキテクチャ設計 (Architecture Design - App & Infra)

- **タスク (IPA):** システム構造、インターフェース、データアーキテクチャの定義⁹。
- 責任分掌による詳細化:
 - **アプリケーションアーキテクトエージェント (AAA):** 論理アーキテクチャを定義する。
 - アクティビティ: ドメイン駆動設計(DDD)に基づき、境界づけられたコンテキスト(Bounded Contexts)を特定し、マイクロサービスの境界を画定する。サービス間の通信プロトコル(REST, gRPC, GraphQL)とAPIスキーマ(OpenAPI/Swagger)を策定する。
 - 成果物: api-spec.yaml (OpenAPI Spec), domain-model.md, service-boundary.mermaid.
 - **インフラストラクチャアーキテクトエージェント (IAA):** 物理・クラウドアーキテクチャを定義する。
 - アクティビティ: AAAが定義した論理サービスを、実際のクラウドリソースにマッピングする(例:「決済サービス」→「AWS Lambda + API Gateway」)。VPC設計、サブネット分割、ルートテーブル、セキュリティグループ、IAMロールの最小権限原則に基づく設計を行う。
 - 成果物: infra-topology.tf (Terraform), security-groups.json, architecture-diagram.py (Diagrams as Code).

4.2.2 第3階層: インターフェース設計 (Interface Design)

- **タスク (IPA):** 外部および内部インターフェースの設計。
- **AIワークフロー:**
 - **AAA:** サービスの「契約(Contract)」を生成する。入力パラメータ、出力形式、エラーハンドリングを厳密に定義する。
 - **PCA(後続):** この契約に従って実装を行う義務を負う。
 - **IAA:** ネットワークレベルでこの契約が可能であることを保証する(例: ポート443の開放、API GatewayのCORS設定、VPC Peeringの確立)。

4.3 IPA 第2階層: ソフトウェア実装 (Software Implementation)

設計に基づき、実際に動作するソフトウェアとインフラを構築するフェーズである。ここでは、動的に生成されたポリグロットコーダーが活躍する。

4.3.1 第3階層: プログラミング (Programming / Coding)

- **タスク (IPA):** コード生成、単体テスト、デバッグ。
- **AIワークフロー:**
 - **ポリグロットコーダーエージェント (PCA):** このエージェントは動的にインスタンス化される。アーキテクチャ設計が「Pythonバックエンド」を指定していれば、メタアーキテクトは「Pythonコーダー」を生成する。もし「Rustマイクロサービス」であれば、「Rustコーダー」が生成される。
 - **入力:** api-spec.yaml (AAA作成) および infra-config.json (IAA作成。データベース接続文字列のプレースホルダーなどを含む)。
 - **アクション:** 定義された仕様に基づきソースコードを記述する。
 - **制約:** PCAはアーキテクチャを変更する権限を持たない。もし新しいライブラリが必要になった場合、PCAは勝手に追加するのではなく、AGENTS.md または依存関係ファイルの更新をリクエストし、AAAによるレビュー(再設計)をトリガーしなければならない¹³。これにより、開発現場の独断によるアーキテクチャの崩壊を防ぐ。

4.3.2 第3階層: インフラ構築 (Infrastructure Construction)

- **タスク (IPA):** ハードウェア・ソフトウェア環境の構築、設定。
- **AIワークフロー:**
 - **IAA:** IaCスクリプト(Terraform/CDK)を実行し、クラウド環境をプロビジョニングする。
 - **セキュリティレビュー:** デプロイ前に、Checkov や tfsec などのMCPツールを使用して、生成されたインフラコードのセキュリティスキャンを実行する¹⁹。コンプライアンス違反(例: S3バケットのパブリック公開)が検出された場合、IAAは自動的に修正案を作成し、再レビューを行う。

4.3.3 第3階層: テスト (Testing)

- **タスク (IPA):** 単体テスト、結合テストの実施。
- **AIワークフロー:**
 - **テストエージェント:** PCAが作成したコードに対して、単体テスト(Unit Test)を自動生成・実行する。AAAが定義した契約に基づき、APIの契約テスト(Contract Testing)も実施する。
 - **自己修復:** テストが失敗した場合、PCAはエラーログを解析し、コードを修正して再テストを行うループに入る。

5. 詳細サブエージェント仕様とスキルマトリクス

ここでは、ユーザーの要求に基づき、詳細化されたサブエージェントの定義と、それらに求められるスキルセットをマトリクス形式で提示する。このマトリクスは、IPAプロセスの各フェーズにおいて、どのエージェントがどのような能力(MCPツール)を行使し、どの程度の人間による監視(Human Oversight)を必要とするかを定義している。

表1: 戦略・企画フェーズ (Strategy & Planning Phase - IPA Layer 2 & 3)

IPA 第2 階層プロセス	IPA 第3 階層アクティビティ	エージェント役割	サブエージェント識別子	必須スキル / MCP ツール能力	動的構成 トリガー (新技術 対応)	人間による監視 (Human Oversight)
システム 企画	要件分析	要件分析 エージェント	Agent_An alyst_Am biguity	NLP/推 論: 曖昧 性検出 ²² 、ユー ザース トーリー マッピング。 ツール: mcp-jira, mcp-con fluence.	新しい法 規制(例: EU AI Act, GDPR改 定)に基 づくプロ ンプト戦 略の更新。	高: 要件 定義書(RDD)の レビュー と承認。
	技術ス タック選 定	メタアー キテクト	Agent_Fa ctory_Ma ster	知識検索: Tech Radar分 析、互換 性チェッ ク。 ツール: mcp-web -search, mcp-vect or-store.	新しい言 語バー ジョン(例 : Java 21 LTS)やフ レーム ワークの 登場を検 知し、コー ダー用テ ンプレート を更新。	中: 選定 された技 術スタック の承認。
	プロジェ クト計画	PM エー ジェント	Agent_Sc rum_Mas ter	計画: WBS生 成、工数 見積もり。 ツール: mcp-line ar,	過去のベ ロシティ データに 基づく見 積もりロ ジックの 自己補正 (	中: スケ ジュール とマイル ストーン の確認。

				mcp-excel.	Self-Correction)。	
--	--	--	--	------------	-------------------	--

表2: アーキテクチャ・設計フェーズ (Architecture & Design Phase)

IPA 第2 階層プロセス	IPA 第3 階層アクティビティ	エージェント役割	サブエージェント識別子	必須スキル / MCP ツール能力	動的構成 トリガー (新技術 対応)	人間による監視 (Human Oversight)
システム 設計	論理アーキテクチャ設計	アプリアーキテクト (AAA)	Agent_Arch_DDD	設計パターン: マイクロサービス, イベント駆動 ⁵ , DDD。 ツール: mcp-diagrams (PlantUML/Mermaid). ²¹	新しい設計パラダイム(例: Data Mesh)のシステムプロンプトへの統合。	高: アーキテクチャレビュー委員会 (ARB) による承認。
	データモデリング	アプリアーキテクト (AAA)	Agent_Data_Modeler	DB理論: 正規化, NoSQLスキーマ設計。 ツール: mcp-postgres, mcp-pinecone. ¹⁶	新しいDB機能(例: PostgresのVectorサポート)のプロンプト更新。	中: データモデルとスキーマのレビュー。
	物理/クラウド設計	インフラアーキテ	Agent_Cloud_AWS	IaCエキスパート: Terrafor	重要: AWS/Azureの新	高: セキュリティ監査とコスト試

		クト (IAA)		m, CDK, CloudFormation。 ツール: aws-iac-mcp, terraform-mcp. ¹²	サービス追加や廃止情報の週次更新。	算の確認。
	セキュリティ設計	インフラアーキテクト (IAA)	Agent_Sec_Engineer	サイバーセキュリティ: IAM, VPC設計, 暗号化標準 ⁷ 。 ツール: mcp-sentry, checkov.	ゼロデイ脆弱性パッチ情報のルールベースへの即時反映。	高: セキュリティサインオフ。

表3: 実装・構築フェーズ (Implementation & Construction Phase)

IPA 第2 階層プロセス	IPA 第3 階層アクティビティ	エージェント役割	サブエージェント識別子	必須スキル / MCP ツール能力	動的構成トリガー (新技術対応)	人間による監視 (Human Oversight)
ソフトウェア実装	プログラミング(ロジック)	ポリグロットコーダー (PCA)	Agent_Coder_Lang_X	構文エキスパート: Python/Rust/JS 等の言語仕様。 ツール: mcp-filesystem, mcp-gith	動的生成: 設計で定義された言語ごとに生成される。最新のライブラリAPIへの追従。	低: PRLレビュー (AIレビューア + 人間)。

				ub.		
	単体テスト	テストエージェント	Agent_Q A_Unit	テストフレームワーク: PyTest, JUnit, Jest。 ツール: mcp-files system (read-only code).	新しいテストライブラリ(例: Vitest vs Jest)の採用。	低: 自動テストの Pass/Fail 確認。
	インフラ構築	インフラアーキテクト (IAA)	Agent_De vOps_De ploy	CI/CD: GitHub Actions, Jenkins パイプライン。 ツール: mcp-github-actions, mcp-docker.	CIランナーイメージの更新やデプロイ戦略(Blue/Green)の変更。	中: デプロイゲートでの承認。
	コードレビュー	レビューエージェント	Agent_Re viewer_St atic	静的解析: Linting, SonarQube ルール。 ツール: ndlovu-code-reviewer.²⁰	新しい Linter ルールの動的な注入。	中: 最終マージ承認。

6. プロセス設計: 動的ライフサイクルワークフローの詳細

本セクションでは、IPAフレームワークに沿ってエージェントがどのように協調し、動的生成メカニズムがどのように機能するかを、時系列のナラティブとして記述する。

6.1 フェーズ1: 動的エージェントのブートストラップ(ビッグバン)

プロジェクトの開始前、つまりIPAの「システム企画」に入る前に、メタアーキテクトによる環境の初期化が行われる。

1. トリガー: ユーザーからの入力(例:「高頻度取引(HFT)プラットフォームを構築したい。低遅延が最優先である」)。
2. 分析: メタアーキテクトはこの要求を解析する。「高頻度取引」というキーワードから、レイテンシ要件が極めて厳しいことを認識する。
3. エージェント選定:
 - アプリケーションアーキテクト:「イベント駆動アーキテクチャ」および「低遅延設計」のペルソナを持つバージョンを選択。
 - インフラアーキテクト:「ベアメタル」または「FPGA/GPUアクセラレーション」の知識を持つ AWS/On-Premise/ハイブリッド構成のペルソナを選択。
 - コーダー: 一般的なPythonではなく、「Rust」および「C++」のプロファイルを持つエージェントを選択する。ガベージコレクション(GC)による停止を許容できないため、GoやJavaは除外される。
4. インスタンス化: メタアーキテクトは、これらの決定に基づき、具体的な AGENTS.md 構成ファイル¹³を生成する。
 - 記述例:「あなたはRustコーダーである。非同期処理には tokio を使用せよ。メモリ安全性に厳格であれ。clippy の pedantic ルールに準拠せよ。」

6.2 フェーズ2: 協調的設計(ハンドシェイク)

このフェーズはIPAの「システム設計」に相当する。AAとIAの対話が中心となる。

1. アプリケーションアーキテクト(AA)のアクション:
 - api-schema.graphql(API仕様)をドラフトする。
 - component-diagram.mermaid(コンポーネント図)を作成する²¹。
 - ハンドオフ: これらの成果物をGitリポジトリにコミットし、IAに通知する。
2. インフラアーキテクト(IA)のアクション:
 - component-diagram.mermaid を読み込む。
 - 推論:「AAはRedisクラスタによるキャッシングを要求している。また、HFTのために専用の Direct Connectが必要だ。」
 - アクション: Redisクラスタをプロビジョニングするための redis.tf (Terraform) を生成する。
 - 出力: 将来のリソースエンドポイント(Redisのホスト名など)を含む config.env.template を生成し、AAおよびコーダーに提供する。
3. 紛争解決(メタエージェントの介入):
 - もしAAが物理的に不可能な要求(例:「存在しないAWSリージョンでのサービス展開」や「光速を超えるレイテンシ要求」)をした場合、IAはエラーを報告する。メタエージェントはこの対

立を調停し、AAIに対して設計の修正(リビジョン)を指示する。

6.3 フェーズ3: ポリグロット実装(スワーム)

IPAの「プログラミング」および「構築」フェーズである。ここでは多数のコーダーエージェントが並行して作業を行う。

1. 動的生成: メタアーキテクトは設計仕様書を読み込む。そこには3つのサービスが定義されている:「注文処理サービス(Rust)」、「管理画面(React)」、「分析サービス(Python)」。
2. **Spawn**(生成): メタアーキテクトは、それぞれの言語に特化した3つの異なるポリグロットコーダーエージェント (**PCA**) を生成する。それぞれのエージェントには、その言語の最新のドキュメントとベストプラクティス(Linter設定など)がロードされる²⁵。
3. 実行:
 - Rustエージェント: api-schema.graphql と config.env を読み込み、main.rs を実装する。
 - Reactエージェント: api-schema.graphql を読み込み、App.tsx を実装する。
4. 自己修復ループ:
 - Rustエージェントがコンパイルエラーに遭遇した場合、内部で修正ループを回す。
 - もし「アーキテクチャ上の欠陥」(例: API仕様に必要なフィールドが欠けている)を発見した場合、PCAは勝手にフィールドを追加するのではなく、作業を一時停止(Pause)し、AAエージェントをタグ付けして修正を依頼する。これにより、実装者が勝手に仕様を変えることを防ぎ、責任分掌を維持する。

6.4 フェーズ4: 進化と更新(エバーグリーン)

システムが新技術にどのように対応するかを示すシナリオである。

1. イベント: AWSがJava Lambda向けの「SnapStart」機能の正式サポートを発表する。
2. 検知: テクノロジーウォッチャーエージェント(定期実行)が、mcp-firecrawl を用いてAWS公式ブログやリリースノートをスクレイピングし、この情報を検知する¹⁵。
3. 更新: ウォッチャーはメタアーキテクトに通知する。
4. リファクタリング: メタアーキテクトは、インフラアーキテクトエージェントのシステムプロンプト(または AGENTS.md のインフラセクション)を更新する。「Java Lambdaをデプロイする際は、パフォーマンス向上のためSnapStartを有効化することを推奨する」というルールを追加する。
5. 波及効果: 次回、IAが terraform plan を実行する際、既存のJava Lambdaリソースに対して SnapStartを有効化する変更案を提示する。これにより、システムは自動的にモダナイズされる。

7. 技術的実装戦略: MCPとファイルシステム

7.1 「生きている構成ファイル」システム (Living Configuration System)

動的生成を支えるのは、静的なコードではなく「生きている構成ファイル」である。

- **AGENTS.md**: エージェントの「DNA」として機能する階層化されたMarkdownファイル¹³。

- *Root*: プロジェクト全体の方針 (例:「ログは全てJSON形式で出力せよ」)。
- *Sub-folder (infra/)*: インフラ特有のルール (例:「S3バケットのパブリックアクセスは禁止」)。
- *Sub-folder (src/)*: コーディング規約 (例:「Try/Catchブロックを必須とする」)。
- **Vector Store (RAG)**: エージェントが学習した教訓を保存する「長期記憶」。もしRustエージェントが複雑な並行処理のバグを修正した場合、その解決策を「ケーススタディ」としてVector Storeに書き込む。将来生成されるRustエージェントは、類似のタスクに直面した際、この記憶を参照する。

7.2 MCPサーバーエコシステム

本アーキテクチャの実装は、以下の特定のMCPサーバー群に依存している¹²。これらはエージェントに「手足」を与える。

MCP Server	アーキテクチャ内での用途
@modelcontextprotocol/server-filesystem	エージェントがコードを編集し、設定ファイルを読み込み、AGENTS.mdを管理するために必須。
awslabs/aws-iac-mcp-server	インフラアーキテクト (IA) に、有効なCloudFormation/CDKコンストラクトとセキュリティチェック (cdk-nag) の能力を提供する ¹² 。
awslabs/terraform-mcp-server	IAがTerraformの plan/apply を実行し、Checkovによるスキャンを行うために使用 ¹⁹ 。
ndlovu-code-reviewer	レビューエージェントに、Gitのdiff解析能力とLinter連携機能を提供する ²⁰ 。
mcp-diagrams	アプリケーションアーキテクト (AA) が、システム構造を可視化し、人間による検証を容易にするために使用 ²¹ 。
mcp-git / mcp-github	コードのバージョン管理、ブランチ作成、PR作成を行う。開発フローの根幹。

7.3 セキュリティガードレール

「エージェントスワーム」が暴走し、セキュリティリスクとならないよう、以下のガードレールを強制する⁷。

1. 最小権限の原則 (**Least Privilege**): コーダーエージェント (PCA) は、AWSのクレデンシャル

(認証情報)に一切アクセスできない。彼らは config.env のプレースホルダーしか見ることができない。デプロイ権限を持つのはインフラアーキテクト (IA) のみであり、IAはコードを書かない。この分離により、悪意あるコードがインフラを操作するリスクを排除する。

2. **Human-in-the-Loop** ゲート: IPA第3階層の重要な移行ポイント(設計→実装、実装→本番環境)においては、MCPの「Sampling(サンプリング)」または「Elicitation(聞き取り)」プリミティブを使用し、人間による明示的な承認トークンを要求する¹⁷。
3. 静的解析の関門: textlint や ndlovu-code-reviewer などのMCPチェックを通過しないコードは、リポジトリへのコミットが拒否される²⁶。

8. 結論と展望

本報告書で提案したアーキテクチャは、IPA共通フレームという堅牢なプロセスモデルに、最新の **Model Context Protocol (MCP)** と自己進化型AIエージェントの技術を統合したものである。

アプリケーション、インフラ、コーディングの役割を厳格に分離することで、AIのコンテキストウィンドウの制限を克服し、幻覚のリスクを最小化した。また、メタアーキテクトによる動的なエージェント生成メカニズムは、技術の陳腐化という課題に対する根本的な解決策を提供する。システムは固定されたツールセットに縛られることなく、常に最新の技術トレンドに合わせて自身の「労働力」を再構成し続けることができる。

このアーキテクチャは、単なる自動化ツールではなく、企業のソフトウェア開発能力を「運用可能な資産」として再定義するものである。今後の展望として、このシステムが蓄積する「成功したアーキテクチャパターン」と「失敗の教訓」は、企業固有の知的財産となり、競争優位性の源泉となるであろう。

補足分析: 研究資料に基づく洞察

1. 研究資料の分析

提供された研究資料は、本アーキテクチャの構築において強固な基盤となった。

- **IPA共通フレーム (SLCP):**⁹ は、エージェントのワークフローを構造化するための分類法(システム企画、システム設計、プログラミング)を提供した。特に「フェーズ内評価 (In-Phase Assessment)」²⁸ の概念は、Human-in-the-Loopゲートの設計において決定的であった。
- **責任分掌:**⁵ は、現代アーキテクチャにおける責任分掌の必要性を補強した。「レイヤードアーキテクチャ」や「マイクロサービス」パターンは、AA/IA/PCAの分離モデルに強く影響している。
- **動的エージェントとMCP:**「エージェント・スミス」パターン¹⁰、「自己進化型エージェント」²、および詳細なMCPドキュメント¹¹ は、動的生成のメカニズムを提供した。AGENTS.md の概念¹³ は、ファイルシステムアクセスを通じたエージェント指示の具体的かつ実行可能な手法であった。
- **セキュリティ:** AWSセキュリティのベストプラクティス⁷ は、インフラエージェントのペルソナに統合された。

2. アーキテクチャ上の決定理由

- **なぜAAとIAを分離するのか?** クラウドネイティブ時代において「インフラはコード」であるが、複

雑なビジネスロジック(例: 保険引受ルール)と複雑なインフラロジック(例: AWS VPCピアリング、サブネットマスキング)を単一のエージェントに処理させると、コンテキストウィンドウが溢れ、推論能力が低下する。分離することで、特化したコンテキストロード(Appエージェントはドメイン文書を、InfraエージェントはAWS文書をロード)が可能となる。

- なぜ動的生成なのか？ 標準的な「コーディングエージェント」は脆い場合が多い。タスクに特化してコーダーエージェントを生成するメタアーキテクトを持つことで(例:「高並行処理 tokio 使用に特化したRustエージェントを作成せよ」)、システムプロンプトのノイズを減らせる。エージェントはそのタスクに必要なことだけを知っていればよい。
- なぜMCPなのか？ MCPはAIにとっての業界標準「USBポート」である。ベンダーロックインを防ぎ、エージェントがツールを切り替える(例: 汎用ファイルIOから特化したGitHub統合へ)際に、エージェントのコアを書き換える必要がないためである。

3. 第2・第3次オーダーの洞察

- 洞察1(「生きているドキュメント」の波及効果): エージェントが機能するために AGENTS.md やドキュメントに依存する場合、ドキュメントの品質がコード品質の主要な制約要因となる。本アーキテクチャにおいて、優れたドキュメントを書くことはもはや人間への説明のためだけでなく、エージェントへの「ソースコード」となる。これは「テクニカルライター」の役割を、極めて重要なエンジニアリング機能へと昇格させる。
- 洞察2(「ジュニア」ロールの陳腐化): 「ポリグロットコーダー」エージェントは、通常ジュニア開発者が行う実装作業を効果的に代替する。人間の役割は完全に「レビューア」「アーキテクト」「プロンプター」へとシフトする。これは育成上のギャップを生む。ジュニアコーダーとしての経験がないまま、どうやってシニアアーキテクトを育成するのか？ 本アーキテクチャは専門家による監視を前提としており、高度に熟練した人間のオペレーターが必要であることを示唆している。
- 洞察3(コードとしてのガバナンス): 「メタエージェント」には企業のガバナンスポリシー(例:「GCPIは禁止、AWSのみ使用」)をプログラムできる。これにより、サブエージェントが何を「したい」と思っても、ファクトリーはポリシーに違反する能力を持つエージェントを生成すること自体を行わない。ガバナンスは事後のチェックではなく、生成的な制約となる。

引用文献

1. Auto-scaling LLM-based multi-agent systems through dynamic integration of agents - Frontiers, 12月 29, 2025にアクセス、
<https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1638227/full>
2. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? - arXiv, 12月 29, 2025にアクセス、<https://arxiv.org/html/2511.13646v1>
3. Daily Papers - Hugging Face, 12月 29, 2025にアクセス、
<https://huggingface.co/papers?q=self-evolving%20agent%20system>
4. Self-Evolving Agents - Emergent Mind, 12月 29, 2025にアクセス、
<https://www.emergentmind.com/topics/self-evolving-agents>
5. 10 Software Architecture Patterns You Must Know About in 2025 - Clustox Blog, 12月 29, 2025にアクセス、
<https://www.clustox.com/blog/software-architecture-patterns/>

6. 10 Essential Software Architecture Best Practices for 2025 | 42 Coffee Cups Blog, 12月 29, 2025にアクセス、
<https://www.42coffeecups.com/blog/software-architecture-best-practices>
7. 10 AWS Security Best Practices Every Team Should Know in 2025. - Jeevi Academy, 12月 29, 2025にアクセス、
<https://www.jeeviacademy.com/10-aws-security-best-practices-every-team-should-know-in-2025/>
8. AWS CDK security best practices - AWS Documentation, 12月 29, 2025にアクセス、
<https://docs.aws.amazon.com/cdk/v2/guide/best-practices-security.html>
9. 2-2 Task Dictionary structure - IPA, 12月 29, 2025にアクセス、
<https://www.ipa.go.jp/en/it-talents/skill-standard/gg62ps0000000kno-att/000061798.pdf>
10. Agent system design patterns | Databricks on AWS, 12月 29, 2025にアクセス、
<https://docs.databricks.com/aws/en/generative-ai/guide/agent-system-design-patterns>
11. Guide - Model Context Protocol (MCP), 12月 29, 2025にアクセス、
<https://modelcontextprotocol.info/docs/quickstart/guide/>
12. Introducing the AWS Infrastructure as Code MCP Server: AI-Powered CDK and CloudFormation Assistance | AWS DevOps & Developer Productivity Blog, 12月 29, 2025にアクセス、
<https://aws.amazon.com/blogs/devops/introducing-the-aws-infrastructure-as-code-mcp-server-ai-powered-cdk-and-cloudformation-assistance/>
13. How to teach your coding agent with AGENTS.md - Eric J. Ma's Personal Site - Eric Ma, 12月 29, 2025にアクセス、
<https://ericmjl.github.io/blog/2025/10/4/how-to-teach-your-coding-agent-with-agentsmd/>
14. AGENTS.md, 12月 29, 2025にアクセス、
<https://agents.md/>
15. Awesome MCP Servers, 12月 29, 2025にアクセス、
<https://mcpservers.org/>
16. The Ultimate Model Context Protocol Directory - MCP Server Finder, 12月 29, 2025にアクセス、
<https://www.mcpserverfinder.com/servers>
17. Architecture overview - Model Context Protocol, 12月 29, 2025にアクセス、
<https://modelcontextprotocol.io/docs/learn/architecture>
18. Use MCP servers in VS Code, 12月 29, 2025にアクセス、
<https://code.visualstudio.com/docs/copilot/customization/mcp-servers>
19. AWS Terraform MCP Server, 12月 29, 2025にアクセス、
<https://awslabs.github.io/mcp/servers/terraform-mcp-server>
20. ndlovu-code-reviewer - Awesome MCP Servers, 12月 29, 2025にアクセス、
<https://mcpservers.org/servers/github-com-shaunie2fly-ndlovu-code-reviewer>
21. A Deep Dive into mcp-diagram: The MCP Server for AI Engineers - Skywork.ai, 12月 29, 2025にアクセス、
<https://skywork.ai/skypage/en/A%20Deep%20Dive%20into%20mcp-diagram%3A%20The%20MCP%20Server%20for%20AI%20Engineers/1970684930865295360>
22. Bridging Precision and Complexity: A Novel Machine Learning Approach for Ambiguity Detection in Software Requirements - IEEE Xplore, 12月 29, 2025にアクセス、
<https://ieeexplore.ieee.org/iel8/6287639/10820123/10843220.pdf>

23. AI in Requirements Management: Techniques, Process and Tools - Visure Solutions, 12月 29, 2025にアクセス、
<https://visuresolutions.com/alm-guide/ai-requirements-management/>
24. Terraform Meets DevSecOps: 5 Security Practices You Can't Afford to Ignore - DEV Community, 12月 29, 2025にアクセス、
<https://dev.to/aws-builders/terraform-meets-devsecops-5-security-practices-you-cant-afford-to-ignore-1n91>
25. 5 Essential MCP Servers Every Developer Should Know, 12月 29, 2025にアクセス、
<https://medium.com/@riccardo.tartaglia/5-essential-mcp-servers-every-developer-should-know-72e828cae18e>
26. textlint MCP Server Setup, 12月 29, 2025にアクセス、<https://textlint.org/docs/mcp/>
27. SLCP process: what is the assessment process like?, 12月 29, 2025にアクセス、
<https://slcp.zendesk.com/hc/en-us/articles/6275035409564-SLCP-process-what-is-the-assessment-process-like>
28. In-Phase Assessment Process Guide, 12月 29, 2025にアクセス、
https://www.pa.gov/content/dam/copapwp-pagov/en/dhs/documents/providers/providers/documents/business-and-tech-standards/business-domain/p_031877.doc